Lab Ten – Point Estimation

Tips

- Complete the tasks below. Make sure to start your solutions in on a new line that starts with "**Solution**:".
- Make sure to use the Quarto Cheatsheet. This will make completing and writing up the lab *much* easier.
- Make sure to use "enter" to make your code readable, so it doesn't run off the page. I switched the Quarto document to automatically render to .pdf, so you can see what you're submitting by default. You can switch back for the highlighting by moving the html block above the pdf block in the YAML header.
- Don't forget that you can copy-and-paste code when you're doing something similar as we did in class!
- Make sure to plot all computed probabilities. This is good `ggplot` practice AND will help make sure you don't make a mistake when computing the probability.

1 Lab 8 Notes

1.1 Altham's Multiplicative Binomial Distribution

The PMF for Altham's Multiplicative Binomial Distribution is

$$f_X(x) = \frac{1}{c}\binom{n}{x} p^x (1-p)^{n-x} \theta^{x(n-x)} I(x \in \{0, 1, ..., n\})$$

where

$$c = \sum_{i=0}^{n} \binom{n}{i} p^i (1-p)^{n-i} \theta^{i(n-i)}.$$

Just so we start on the same page, I have included my `R` function for computing probability using this distribution.

```r
daltbinom <- function(x, size, prob, theta){
  normalizing.constant <- 0
  for(j in 0:size){
    normalizing.constant <- normalizing.constant +
      choose(size, j) * prob^j * (1-prob)^(size-j) * theta^(j*(size-j))
  }
  # Probability Mass at x=x
  (1/normalizing.constant) *
    choose(size, x) * prob^x * (1-prob)^(size-x) * theta^(x*(size-x)) *
      (x>=0)*(x<=size)
}
```

2 Altham's Multiplicative Beta Binomial Distribution

The PMF for Altham's Multiplicative Beta Binomial Distribution is

$$f_X(x) = \left[\int_0^1 \left(\frac{1}{c}\binom{n}{x} p^x (1-p)^{n-x} \theta^{x(n-x)}\right)\left(\frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} p^{\alpha-1}(1-p)^{\beta-1}\right) dp\right] I(x \in \{0, 1, ..., n\})$$

Just so we start on the same page, I have included my `R` function for computing probability using this distribution.

```r
daltbbinom.single <- function(x, size, alpha, beta, theta){
    integrand <- function(p) {
    sapply(p, # integrate will pass in a vector, so we use sapply to apply the following
           function(p_val){
             # f(x|p) * f(p|alpha, beta)
             daltbinom(x, size, p_val, theta) * dbeta(p_val, alpha, beta)
           }
    )
  }
  # Probability Mass at x=x
  # integrate(integrand, lower = 0, upper = 1)$value * (x>=0)*(x<=size)
  # I shrink the bounds below to avoid log(0)=-infinty
  # I use try() to catch when there is an error and avoid crashing
  res <- try(integrate(integrand, lower = 1e-7, upper = 1 - 1e-7)$value *
                 (x>=0)*(x<=size),
             silent = TRUE)
  # If there is an error, return a near-zero probability
  if(inherits(res, "try-error")) return(0.1^6)

  res
}
# Write a function that works on a vector
daltbbinom <- function(x, size, alpha, beta, theta){
  sapply(X=x, FUN=daltbbinom.single, size=size, alpha=alpha, beta=beta, theta=theta)
}
```

3 Question 1

In Lab 8, we use Altham's Multiplicative Binomial Distribution. Your job was to interpret the impact of θ in that equation. In this lab, we will model the number of legs won in a best of eleven (first to six) match.

To help us think about the interpretation, something that was challenging before, let's try something a little more manageable.

3.1 Part a

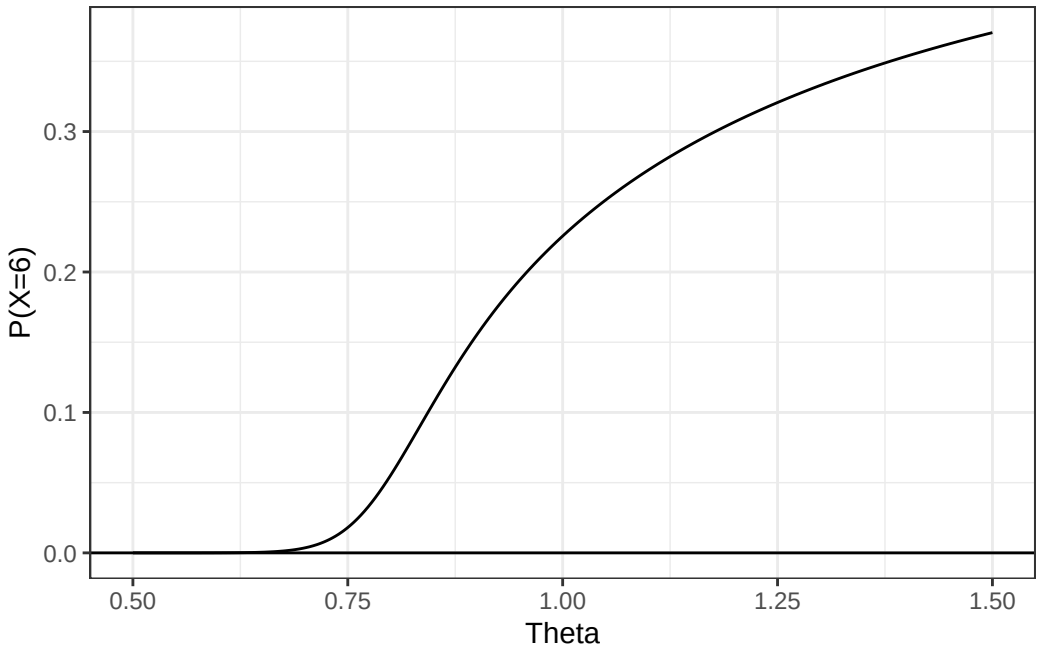
Suppose a player has a 50% chance of winning a specific leg, that is let $p = 0.50$. Let's look at the probability they win ($x = 6$) legs in the match. Compute this probability over a sequence from $\theta = 0.5$ to $\theta = 1.5$ and plot it using a line where the x-axis is θ and the y-axis is $P(X = 6)$.

Solution

```r
ggdat <- tibble(theta = seq(0.5, 1.5, 0.001)) |>
  mutate(Probability = daltbinom(6, size = 11, prob= 0.5, theta = theta))

plot <- ggplot(ggdat) +
  geom_line(aes(x = theta, y= Probability)) +
  geom_hline(yintercept = 0) +
  ylab("P(X=6)") +
  xlab("Theta") +
  theme_bw()

plot
```



3.2 Part b

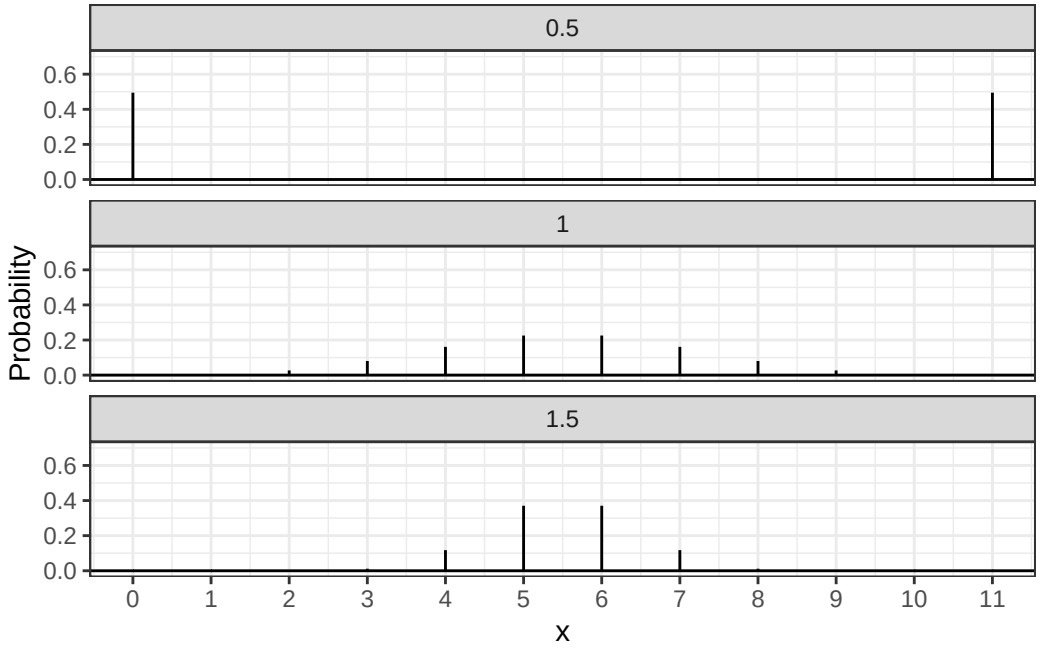
Now, plot Altham's Multiplicative Binomial Distribution under the same parameters with $\theta = 0.50$, $\theta = 1$, and $\theta = 1.5$. Use `ncol=1` in `facet_wrap()` to plot the PMFs in a single column for comparing.

Solurion

```r
ggdat <- tibble(x = rep(seq(0, 11, 1),3), theta = c(rep(0.5, 12),rep(1,12),rep(1.5,12)))|>
  mutate(PMF= daltbinom(x,11,0.5,theta))

PMF.plot <- ggplot() +
  geom_linerange(data = ggdat,
                 aes(x = x, ymin = 0, ymax = PMF)) +
  geom_hline(yintercept = 0) +
  scale_x_continuous("x", breaks=0:11) +
  scale_y_continuous(limits = c(0, 0.7)) +
  ylab("Probability") +
  facet_wrap(~theta, ncol = 1) +
  theme_bw()

PMF.plot
```



3.3 Part c

What is your best assessment of what θ is doing in this setting. Consider when a 6-0 blowout is most likely, and when a 5-6 nail-biter is most likely. Note here we should treat the match as a fixed $n = 11$ as we did for $n = 3$ in Lab 8. **Note:** In lab 8, we saw that small θ meant wins came in bunches (clumped together), so there were more 2-0 wins in a best of three series.

When theta is greater, x, or the number of matches win, are likely to have higher probability at the center.

4 Question 2

4.1 Part a

Altham's Multiplicative Binomial Distribution does not typically simplify to the clean $E(X) = np$ seen in the Binomial distribution, unless $\theta = 1$. Write out the expression that would need to be solved to find $E(X^k)$, and then write a function that computes it given the proposed parameter values p (`prob`) and θ (`theta`).

Solution

$E(X^K) = \sum_{i=0}^{11} x_i^k f_x(x_i)$

```
momdaltbinom <- function(p, theta, k){
  x <- 0:11
  sum(x^k*daltbinom(x,11,0.5,theta))
}
```

4.2 Part b

Describe how you can use your answer to part a to find Method of Moments estimates for the parameters p and θ for Altham's Multiplicative Binomial Distribution based on data.

Solution

We can calculate the $\bar{x}$ and set it equal to $\sum_{i=0}^{11} x_i f_x(x_i)$; calculate $\overline{x^2}$ and set it equal to $\sum_{i=0}^{11} x_i^2 f_x(x_i)$ and solve for p and theta.

4.3 Part c

Write a function that computes the log-likelihood for Altham's Multiplicative Binomial Distribution, given data (`x`) and proposed parameter values p (`prob`) and θ (`theta`).

Solution

```
lldaltbinom <- function(data, p, theta){
  sum(log(daltbinom(data,11,p,theta)))
}
```

4.4 Part d

Describe how you can use your answer to part c to find Maximum Likelihood estimates for the parameters p and θ for Altham's Multiplicative Binomial Distribution based on data.

Solution

Use the function above and solve for the parameter that allows it to achieve maximum.

5 Lab 10

Peter "Snakebite" Wright is a two-time Professional Darts Corporation World Champion (2020, 2022) and a former World No. 1. He is one of the sport's most decorated players, winning nearly 50 PDC titles including the World Matchplay, UK Open, and multiple European Championships.

In 2024, he made The Premier League – a league involving the eight best players that runs every Thursday night from February to May. In this season, he won two of eighteen matches, securing only four points. The spread in legs won was -43, meaning he lost 43 more legs than he won. This performance was rather quite bad. He was soundly trounced in almost all his match ups.

The 2024 standings are below:

Rank	Player
1	Luke Littler
2	Luke Humphries
3	Michael van Gerwen
4	Michael Smith
5	Nathan Aspinall
6	Rob Cross
7	Gerwyn Price
8	Peter Wright

5.1 Summarize the 2024 Data

In `pdc2024.csv`, I have provided the data for the 2024 Premier League season. There are a few data cleaning steps to consider.

- Remove any matches with no `Score` (e.g., a bye or walkover)
- Nights 1 through 16 are best of 11 (first to 6), but the playoffs are extended. For consistency, keep nights 1 through 16 only.
- Make two columns `WinnerLegs` and `LoserLegs` that keep track of the number of legs each player has won

Summarize the data. What do the data tell us about the breakdown of the results?

Solution

```
dat.pdc <- read_csv("pdc2024.csv")

dat.pdc.clean <- dat.pdc |>
  filter(Score != "w/o") |>
```

```
  filter(Night != "Finals") |>
  mutate(WinnerLegs = as.numeric(str_split(Score, "-", simplify = TRUE)[,1]),
         LoserLegs  = as.numeric(str_split(Score, "-", simplify = TRUE)[,2]))
```

```
legwins <- function(name){
  leg <- 0
  for(i in 1:nrow(dat.pdc.clean)){
    if(dat.pdc.clean$Winner[i]==name){
      leg <- leg + 6
    }
    else if(dat.pdc.clean$Loser[i]==name){
      leg <- leg + dat.pdc.clean$LoserLegs[i]
    }
  }
  leg
}

legtotal <- function(name){
  leg <- 0
  for(i in 1:nrow(dat.pdc.clean)){
    if(dat.pdc.clean$Winner[i]==name | dat.pdc.clean$Loser[i]==name){
      leg <- leg + dat.pdc.clean$LoserLegs[i] + dat.pdc.clean$WinnerLegs[i]
    }
  }
  leg
}
```

5.2 Altham's Multiplicative Binomial Distribution

For each player in the 2024 season, use the number of legs won in each match to compute maximium likelihood estimates for p and θ.
Note: The "for each" here can be done more efficiently with a function or a loop!

- Create a vector containing the number of legs won by the player. Note sometimes you will need to pull from winner's legs and sometimes from the loser's legs.
- Find the MLE for the player using this data. Ensure to report p and θ for each player in a nicely formatted table.
  Note: In Lecture on Monday, we used transformations to restrict how `optim()` searches the parameter space. For example, here, you might consider `p <- 1/(1+exp(-par[1]))` and `theta <- exp(par[2])`.
- Make comments about the MLEs in the context of the standings.

```
player.turns <- table(c(dat.pdc.clean$Winner, dat.pdc.clean$Loser))

# This function returns a data frame that contains two column.
# First column is number of legs the player wins in each match,
# and the second column is the total legs that the player played for each match.
Win.each <- function(name){
  total.turn <- as.numeric(player.turns[name])
  win.each <- tibble(Win = rep(NA, total.turn),
                     Total = rep(NA, total.turn))
  n <- 1
  for(i in 1:nrow(dat.pdc.clean)){
    if(dat.pdc.clean$Winner[i]==name){ # If the player is the winner
      win.each$Win[n] <- 6
      win.each$Total[n] <- dat.pdc.clean$LoserLegs[i] + dat.pdc.clean$WinnerLegs[i]
      n <- n + 1
    }
    else if(dat.pdc.clean$Loser[i]==name){ # If the player is the loser
```

```
      win.each$Win[n] <- dat.pdc.clean$LoserLegs[i]
      win.each$Total[n] <- dat.pdc.clean$LoserLegs[i] + dat.pdc.clean$WinnerLegs[i]
      n <- n + 1
    }
  }
  win.each
}

# This function calculates the probability of the player
# wins x time with given probability and theta.
Prob.daltbinom <- function(x, size, prob, theta){
  sapply(1:length(x), function(i){  # Ensure it works on a vector
    xi <- x[i]
    si <- size[i]
    if(xi == 6){  # If the player win the match
      # Calculate the probability and ensuring the last leg is win
      (choose(si-1,5)/choose(si, xi)) * daltbinom(xi, si, prob, theta)
    } else {  # If the player lose the match
      # Calculate the probability and ensuring the last leg is lose
      (choose(si-1, xi)/choose(si, xi)) * daltbinom(xi, si, prob, theta)
    }
  })
}

# This function calculates the log likelihood of different parameters.
# In order to make it easier for optimization, it returns the negative log likelihood.
llProb.daltbinom <- function(par, data){
  p <- 1/(1+exp(-par[1]))  # Ensures probability is between 0 and 1
  theta <- exp(par[2])     # Ensures theta is greater than 0
  x <- data$Win
  size <- data$Total
  ll <- sum(log(Prob.daltbinom(x,size,p,theta)))
  -ll
}

# Create a summary table for the result
MLE.summary <- tibble(Name = names(player.turns),
                      P = NA,
                      Theta = NA)

# Repeat the process for each player
for(i in 1:nrow(MLE.summary)){
  name <- MLE.summary$Name[i]
  win.each <- Win.each(name)
  mle <- optim(par = c(0,0), fn = llProb.daltbinom,
               data = win.each)
  MLE.summary$P[i] <- 1/(1+exp(-mle$par[1]))
  MLE.summary$Theta[i] <- exp(mle$par[2])
}
```

Table 2: P and Theta for each player

Name	P	Theta
Gerwyn Price	0.43	1.12
Luke Humphries	0.58	1.08
Luke Littler	0.59	1.14

Name	P	Theta
Michael Smith	0.50	1.11
Michael van Gerwen	0.50	1.11
Nathan Aspinall	0.51	1.14
Peter Wright	0.24	1.30
Rob Cross	0.46	1.04

5.3 Altham's Multiplicative Beta Binomial Distribution

For each player in the 2024 season, use the number of legs won in each match to compute maxmium likelihood estimates for p and θ.
Note: The "for each" here can be done more efficiently with a function or a loop!

- Create a vector containing the number of legs won by the player. Note sometimes you will need to pull from winner's legs and sometimes from the loser's legs.
- Find the MLEs for the player using this data. Ensure to report α, β, and θ for each player in a nicely formatted table.
  Note 1: Due to the computational complexity of `integrate()`, you should compute the logged probability for `0:6` once and add them according to the data in each player's vector. **Note 2**: In Lecture on Monday, we used transformations to restrict how `optim()` searches the parameter space.
- Plot the underlying Beta(α, β) distribution for each player.
  Note: Add the mean and variance of the Beta to the table of MLEs. Recall

$$E(X) = \frac{\alpha}{\alpha + \beta}$$

$$sd(X) = \sqrt{\frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}}$$

- Make comments about the MLEs in the context of the standings.
  Note: The players alternate who throws first in a leg has the mathematical "head start" to reach a finish before their opponent can take another turn. Players alternate who throws first with each leg.

Solution

```
# This function calculates the probability of the player
# wins x time with given alpha, beta, and theta.
Prob.daltbbinom <- function(x, size, alpha, beta, theta){
  sapply(1:length(x), function(i){  # Ensure it works on a vector
    xi <- x[i]
    si <- size[i]
    if(xi == 6){  # If the player win the match
      # Calculate the probability and ensuring the last leg is win
      (choose(si-1,5)/choose(si, xi)) * daltbbinom(xi, si, alpha, beta, theta)
    } else {  # If the player lose the match
      # Calculate the probability and ensuring the last leg is lose
      (choose(si-1, xi)/choose(si, xi)) * daltbbinom(xi, si, alpha, beta, theta)
    }
  })
}

# This function calculates the log likelihood of different parameters.
# In order to make it easier for optimization, it returns the negative log likelihood.
llProb.daltbbinom <- function(par, data){
  alpha <- exp(par[1])
  beta <- exp(par[2])
  theta <- exp(par[3])
  x <- data$Win
  size <- data$Total
```

```r
  ll <- sum(log(Prob.daltbbinom(x,size,alpha, beta,theta)))
  -ll
}

# Create a summary table for the result
MLE.summary <- tibble(Name = names(player.turns),
                      Alpha = NA,
                      Beta = NA,
                      Theta = NA,
                      Mean = NA,
                      Variance = NA)

# Repeat the process for each player
for(i in 1:nrow(MLE.summary)){
  name <- MLE.summary$Name[i]
  win.each <- Win.each(name)
  mle <- optim(par = c(0,0,0), fn = llProb.daltbbinom,
               data = win.each)
  alpha <- exp(mle$par[1])
  beta <- exp(mle$par[2])
  MLE.summary$Alpha[i] <- alpha
  MLE.summary$Beta[i] <- beta
  MLE.summary$Theta[i] <- exp(mle$par[3])
  MLE.summary$Mean[i] <- alpha / (alpha + beta)
  MLE.summary$Variance[i] <- ((alpha * beta)/((alpha + beta)^2 * (alpha + beta + 1))) ^ (1/2)
}
```

Table 3: Alpha, Beta and Theta for each player

Name	Alpha	Beta	Theta	Mean	Variance
Gerwyn Price	2.58	3.98	1.38	0.39	0.18
Luke Humphries	3.48	2.10	1.36	0.62	0.19
Luke Littler	3.15	1.85	1.50	0.63	0.20
Michael Smith	2.67	2.70	1.41	0.50	0.20
Michael van Gerwen	3.46	3.51	1.34	0.50	0.18
Nathan Aspinall	3.92	3.66	1.37	0.52	0.17
Peter Wright	2.39	9.88	1.56	0.19	0.11
Rob Cross	1.86	2.34	1.36	0.44	0.22

5.4 Executive Summary

Summarize the work you've done here to provide a brief (200-300 word) data-driven summary of the 2024 Premier League season using your work in Lab 10. Your summary should be professional, clear, and all claims should be corroborated with statistical support.